

Allow lambda capture [=, this]

ISO/IEC JTC1 SC22 WG21 P0409R2

To: CWG

Contents

1. [Revision history](#)
2. [Proposal](#)
3. [Impact on the Standard](#)
4. [Formal wording](#)
5. [Alternatives and discussion](#)

Revision history

- [P0409R0](#): Initial proposal.
- [P0409R1](#): Added discussion of alternatives; no change in the proposal proper.
- P0409R2: Add comma after "this", update section numbering for new working draft, change audience from "EWG" to "CWG".

Proposal

We propose to allow [=, this] as a lambda capture.

With the introduction of capture-**this**-by-value in C++17, it may be desirable to increase code readability by being explicit about the kind of capture in the presence of a [=]-default:

```
std::function<void()> X::f(int n) {
    return coin() ? [=, this]() { g(n, x_, y_, z_); }
                  : [=, *this]() { h(n, x_, y_, z_); };
}
```

By contrast, when both [=] and [=, *this] are present in a code base in large numbers, it may be easy to forget that the former is different from the latter: It is hard to notice the absence of something. In such situations, authors may wish to be explicit and to spare the reader from having remember the correct default.

Impact on the Standard

The proposal is a pure core language extension. The newly allowed syntax was previously ill-formed.

Formal wording

In section 8.1.5.2 [expr.prim.lambda.capture], change paragraph 2 as follows.

If a *lambda-capture* includes a *capture-default* that is **&**, no identifier in a *simple-capture* of that *lambda-capture* shall be preceded by **&**. If a *lambda-capture* includes a *capture-default* that is **=**, each *simple-capture* of that *lambda-capture* shall be of the form "**& identifier**", "**this**", or "*** this**". [Note: The form [**&, this**] is redundant but accepted for compatibility with ISO C++ 2014. – end note] Ignoring appearances in initializers of *init-captures*, an identifier or **this** shall not appear more than once in a *lambda-capture*. [Example:

```
struct S2 { void f(int i); };
void S2::f(int i) {
    [&, i]{ };           // OK
    [&, this, i]{ };    // OK, equivalent to [&, i]
```

```

[&, &i]{ };           // error: i preceded by & when & is the default
[=, *this]{ };        // OK
[=, this]{ };         // error: this when = is the defaultOK, equivalent to [=]
[i, i]{ };           // error: i repeated
[this, *this]{ };     // error: this appears twice
}

```

– end example]

Alternatives and discussion

This proposal falls into a very crowded design space where we are left with relatively little room to move. Several alternative changes have been proposed in this space, but by and large they are all mutually exclusive. We present a few plausible alternatives, along with their advantages and disadvantages.

We will not consider deprecation in the following survey, since we are only interested in exploring the long-term direction, and deprecation without direction is not all that interesting. Everything that follows is to be understood in the context of lambdas with default captures.

0. This proposal: Allow [=, this]

Description: [&], [&, this], [=], [=, this] capture **this* by reference; [&, *this], [=, *this] capture **this* by value.

Pros:

- Non-breaking change; this is a pure extension.
- Allows for self-documenting code. Each kind of *this*-capture can be spelled out explicitly, no need to remember defaults.

Cons:

- Adds a redundant syntactical form.

1. Disallow [&, this]

Description: Remove the currently allowed redundant form [&, this]. [&] captures **this* by reference; [&, *this] captures **this* by value.

Pros:

- Removes all redundancy.

Cons:

- Breaking change. However, there is a transition path; equivalent code exists that is valid both with and without this change.
- No self-documenting code. Users need to remember the default. (“Does [=] capture by value or by reference?”)

2. Allow *only* [&, this] and [=, this]

Description: Remove both implicit forms [&] and [=] entirely. [&, this] and [=, this] capture **this* by reference; [&, *this] and [=, *this] capture **this* by value.

Pros:

- Removes all redundancy.
- Leaves only self-documenting code behind.

Cons:

- Breaking change, and no transition path. There is no code that is valid both with and without this change.

Note. This alternative is not viable, since there is no transition path towards it. We mention it mainly because it presents an interesting alternative reality. Moreover, if the present proposal were accepted, this would be a potential and accessible *future* direction.

3. Make [=] capture by value

Description: [=] means [=, *this] and captures **this* by value; [this, =] captures **this* by reference.

Pros:

- Changes the notion of “by value” to something more intuitive.

Prons:

- Code is not so much “self-documenting” as it is perhaps “natural” if you have the appropriate mental model of `this`.

Cons:

- Still allows redundant syntactical forms.
- Breaking change without transition path: `[=]` changes meaning and there is no currently valid equivalent form that preserves meaning.

Note. This direction is popular with several people, since it is “what lambdas should always have been like”. This change is non-atomic, and we do not wish to explore it in detail, but we note in passing that this change contains the present proposal as the non-breaking subset.

Only alternatives 0 and 1 are immediately viable, since there is a transition path from the current standard. Note that alternative 0 (this proposal) is a subset of and starting point for both alternatives 2 and 3. Thus the immediate decision that is required is between 0 and 1. We believe that allowing explicit, unambiguous code (as per 0) adds greater value than removing redundancy from the language (as per 1). Moreover, pragmatically, alternative 1 would perhaps struggle to ever achieve a complete removal of `[&, this]` from the language; at best the feature would end up permanently deprecated. Without pressure to act on the deprecation, its value seems limited. By contrast, alternative 0 enables actionable style rules to “always capture `this` explicitly”, which may add real value to a codebase.

Further discussion

The above alternatives were surfaced during discussion on the reflector and at the BSI meeting in London. We conclude with a few further discussion points that were raised and should be heard.

- “It is special-case behaviour that does not match anything else. [...] It does not add symmetry, it *removes* it.” (Our response: it neither adds nor removes symmetry; it is merely another tool.)
- “I’m afraid readers will think that `[=]` is different from `[=, this]`.” (Our response: Readers may also worry whether `[=]` is different from `[=, *this]`. Codebases should encourage to always be explicit so readers do not need to worry at all.)